Interactive Programming

DM580: Functional Programming

Casper Bach



SDU

Course Plan and Reading

Week 39	Week 40	Week 41	Week 42
Introduction and Haskell Preliminaries *Read:** Foreword + 1 + 2	Types and Basic Functions *Read:* 3 + 4	Lists and Recursion *Read:* 5 + 6 ***Submit handin assn. 1***	*Fall Break*
Week 43	**Week 44**	**Week 45**	**Week 46**
Higher-Order Functions and Type Classes *Read:* 7 + 8 ***Submit handin assn. 2***	Programming with I/O *Read:* 10 ***Submit handin assn. 3***	Monads *Read:* 12	*No lecture* *Read:* TBA ***Submit handin assn. 4***
Week 47	**Week 48**	**Week 49**	**Week 50**
Concurrency *Read:* Beautiful Concurrency	Lazy Evaluation *Read:* 15 ***Submit handin assn. 5***	Exam Project *Read:* TBA	Q&A Session

* Except where otherwise indicated, reading refers to Graham Hutton's *Programming in Haskell*, second edition

Reading Material

This lecture relates to these chapters of *Programming in Haskell, 2nd ed.*

- Chapter 10: Interactive Programming

Learning Objectives

- Explain what a value of type `IO a` represents, and why Haskell programs stay pure despite performing input/output.
- Sequence actions using **`do`** notation, and explain how results flow from one line of a **`do`** block to the next.
- Use `pure` (or `return`) to inject a pure value into an action.
- Write a recursive interactive loop that carries state from iteration to iteration.
- Distinguish `map` from `mapM_` on a list of actions.



Motivation

From Pure to Interactive Programming

- Pure functional programming admit ease of reasoning.
- But many practical programs need side-effects; or become more efficiently executable on current modern hardware if we use them.
- Haskell supports both, via the `IO` monad.
- Before delving into this solution, let us first summarize the problem it solves.

What Goes Wrong with Effects

What Goes Wrong with Effects

- In the presence of effects, the same operation may give different results. E.g.,

What Goes Wrong with Effects

- In the presence of effects, the same operation may give different results. E.g.,

```
x := readLine; y := readLine
```

What Goes Wrong with Effects

- In the presence of effects, the same operation may give different results. E.g.,

```
x := readLine; y := readLine
```

  where `readLine` reads user input, may give two different results `x` and `y`.

What Goes Wrong with Effects

- In the presence of effects, the same operation may give different results. E.g.,

```
x := readLine; y := readLine
```

  where `readLine` reads user input, may give two different results `x` and `y`.

- Sharing changes behavior. E.g.,

What Goes Wrong with Effects

- In the presence of effects, the same operation may give different results. E.g.,

```
x := readLine; y := readLine
```

  where `readLine` reads user input, may give two different results `x` and `y`.

- Sharing changes behavior. E.g.,

```
f(x) * f(x)  ≠  shared := f(x); shared * shared
```

What Goes Wrong with Effects

- In the presence of effects, the same operation may give different results. E.g.,

```
x := readLine; y := readLine
```

  where `readLine` reads user input, may give two different results `x` and `y`.

- Sharing changes behavior. E.g.,

```
f(x) * f(x)  ≠  shared := f(x); shared * shared
```

- Order of evaluation matters. E.g.,

What Goes Wrong with Effects

- In the presence of effects, the same operation may give different results. E.g.,

```
x := readLine; y := readLine
```

  where `readLine` reads user input, may give two different results `x` and `y`.

- Sharing changes behavior. E.g.,

```
f(x) * f(x)  ≠  shared := f(x); shared * shared
```

- Order of evaluation matters. E.g.,

```
print("h"); print("a"); print("h"); print("a")
                        ≠
shared := (print("h"); print("a")); shared; shared
```

Desired Properties of a Solution

Desired Properties of a Solution

- Effects are possible (read/write, state, etc.).

Desired Properties of a Solution

- Effects are possible (read/write, state, etc.).
- Reasoning survives (pure laws still hold, and effectful values can be shared and substituted safely).

Desired Properties of a Solution

- Effects are possible (read/write, state, etc.).
- Reasoning survives (pure laws still hold, and effectful values can be shared and substituted safely).
- Order is explicit (the program text fixes sequencing, not the compiler).

Desired Properties of a Solution

- Effects are possible (read/write, state, etc.).
- Reasoning survives (pure laws still hold, and effectful values can be shared and substituted safely).
- Order is explicit (the program text fixes sequencing, not the compiler).
- Boundaries are clear (pure code cannot accidentally invoke effects).

Desired Properties of a Solution

- Effects are possible (read/write, state, etc.).
- Reasoning survives (pure laws still hold, and effectful values can be shared and substituted safely).
- Order is explicit (the program text fixes sequencing, not the compiler).
- Boundaries are clear (pure code cannot accidentally invoke effects).

Solution: A type "`IO a`" which represents an effectful computation as a *value*.



What is an `IO` Value?

`IO` — an Interface for Building Effectful Programs

`IO` — an Interface for Building Effectful Programs

- `IO a` represents an effectful computation which, if it returns normally (e.g., no erring or non-termination), yields a value of type `a`.

`IO` — an Interface for Building Effectful Programs

- `IO a` represents an effectful computation which, if it returns normally (e.g., no erring or non-termination), yields a value of type `a`.
- Effectful computations of type `IO a` can only be run by the `main` function of a program, or by `ghci`.
  - The `main` functiona always has type `main :: IO ()`.

`IO` — an Interface for Building Effectful Programs

- `IO a` represents an effectful computation which, if it returns normally (e.g., no erring or non-termination), yields a value of type `a`.
- Effectful computations of type `IO a` can only be run by the `main` function of a program, or by `ghci`.
  - The `main` functiona always has type `main :: IO ()`.
- Values can only be built. There is no program of type `IO a → a` that lets us return the value of an `IO` computation.

`IO` — an Interface for Building Effectful Programs

- `IO a` represents an effectful computation which, if it returns normally (e.g., no erring or non-termination), yields a value of type `a`.
- Effectful computations of type `IO a` can only be run by the `main` function of a program, or by `ghci`.
  - The `main` functiona always has type `main :: IO ()`.
- Values can only be built. There is no program of type `IO a → a` that lets us return the value of an `IO` computation.
- Values are composed from IO operations such as
  - `getChar :: IO Char`
  - `getLine :: IO String`
  - `putChar :: Char → IO ()`
  - `putStr, putStrLn :: String → IO ()`

`IO` Values

- We can pass around `IO` computations as values inside pure programs.
- E.g., the following is a perfectly well-typed program of type `[IO ()]`, which has no observable side effects when run (try yourself in `ghci`)

```
let greeting = putStrLn "Hello world!" in [greeting]
```

- What do the following programs do when called from `ghci`?
  - `putStrLn "foo"`
  - `let x = putStrLn "bar" in x`
  - `Just (putStrLn "baz")`
  - `getLine`



Do Notation

do Notation for Sequencing Computations

```
main :: IO ()
main = do
  putStrLn "What is your name?"
  name ← getLine
  putStrLn ("Hello, " ++ name)
```

- Line breaks followed by a line at the same indentation level represents a new command (corresponding to a semicolon).
- `x ← action` binds the result value of the computation action to x.
- Just action (without "x ←" in front) runs action and then discards its result.

do Blocks as Values

do Blocks as Values

```
nameAndAge :: IO (String, a)
nameAndAge = do
  n ← getLine
  a ← getLine
  pure (n, read a)
```

do Blocks as Values

```
nameAndAge :: IO (String, a)
nameAndAge = do
  n ← getLine
  a ← getLine
  pure (n, read a)

main :: IO ()
main = do
  (n, a) ← nameAndAge
  putStrLn ("Name: " ++ n)
  putStrLn ("Age:  " ++ show a)
```

Equational Rewriting with IO

Equational Rewriting with IO

```
do (n, a) ← nameAndAge
   putStrLn ("Name: " ++ n)
   putStrLn ("Age:  " ++ show a)
```

Equational Rewriting with IO

```
do (n, a) ← nameAndAge
   putStrLn ("Name: " ++ n)
   putStrLn ("Age:  " ++ show a)

== { expand nameAndAge }
```

Equational Rewriting with IO

```
do (n, a) ← nameAndAge
   putStrLn ("Name: " ++ n)
   putStrLn ("Age:  " ++ show a)

= { expand nameAndAge }

do (n, a) ← (do n ← getLine
                a ← getLine
                pure (n, read a))
   putStrLn ("Name: " ++ n)
   putStrLn ("Age:  " ++ show a)
```

Equational Rewriting with IO

```
do (n, a) ← nameAndAge
   putStrLn ("Name: " ++ n)
   putStrLn ("Age:  " ++ show a)

= { expand nameAndAge }

do (n, a) ← (do n ← getLine
                a ← getLine
                pure (n, read a))
   putStrLn ("Name: " ++ n)
   putStrLn ("Age:  " ++ show a)

= { merge do (using laws of monadic bind – next week!) }
```

Equational Rewriting with IO

```
do (n, a) ← nameAndAge
   putStrLn ("Name: " ++ n)
   putStrLn ("Age:  " ++ show a)

= { expand nameAndAge }

do (n, a) ← (do n ← getLine
                a ← getLine
                pure (n, read a))
   putStrLn ("Name: " ++ n)
   putStrLn ("Age:  " ++ show a)

= { merge do (using laws of monadic bind – next week!) }

do n ← getLine
   a ← getLine
   putStrLn ("Name: " ++ n)
   putStrLn ("Age:  " ++ show a)
```

pure / return

`pure` / `return`

- While we cannot “exit” the `IO` monad (no plain function of type `IO a → a`), it is useful and necessary to “enter” it.

`pure` / `return`

- While we cannot “exit” the `IO` monad (no plain function of type `IO a → a`), it is useful and necessary to “enter” it.
- `pure :: a → IO a` takes a value of type `a` as input, and constructs a computation that, when run, returns that value.

`pure` / `return`

- While we cannot “exit” the `IO` monad (no plain function of type `IO a → a`), it is useful and necessary to “enter” it.
- `pure :: a → IO a` takes a value of type `a` as input, and constructs a computation that, when run, returns that value.
- Book uses `return :: a → IO a` for its programs. For `IO` they are aliases.

`pure` / `return`

- While we cannot “exit” the `IO` monad (no plain function of type `IO a → a`), it is useful and necessary to “enter” it.
- `pure :: a → IO a` takes a value of type `a` as input, and constructs a computation that, when run, returns that value.
- Book uses `return :: a → IO a` for its programs. For `IO` they are aliases.

```
pure = return
```

`pure` / `return`

- While we cannot “exit” the `IO` monad (no plain function of type `IO a → a`), it is useful and necessary to “enter” it.
- `pure :: a → IO a` takes a value of type `a` as input, and constructs a computation that, when run, returns that value.
- Book uses `return :: a → IO a` for its programs. For `IO` they are aliases.

```
pure = return
```

- Modern Haskell tends to favor the alias `pure` over `return`.

The Boundary is the Point

Checking our solution against our earlier requirements

- **Effects are possible:** E.g., `putStrLn`, `getLine`, etc.
- **Reasoning survives:** An `IO a` value is a description of a computation, not a performed effect.

```
let x = do putStr "h"; putStr "a" in do x; x
  -- prints "haha"
```

- **Order is explicit. `do`** notation fixes order in the program text. The compiler knows not to reorder `IO` values.
- **Boundaries are clear.** Actions are run only by `main` or `ghci`; there is no function `IO a → a`. Pure code cannot accidentally trigger an effect.



Mapping Over A List of Actions

Recall `map :: (a → b) → [a] → [b]`

- Say we want to visit a list of elements and perform an effectful computation for each
- We might usuccessfully try the following. What would its type be?

```
map putStrLn ["a", "b"]
```

- Prelude provides two functions tailored for this use case (for any monad, but specialized to IO for now)
  - `mapM  :: (a → IO b) → [a] → IO [b]`
  - `mapM_ :: (a → IO ()) → [a] → IO ()`

Exercise: Implement mapM

```
mapM :: (a → IO b) → [a] → IO [b]
mapM f xs = _
```



A Test Runner Using `IO`

Let’s Build a Test Runner

A First, Suboptimal Attempt

```
type Test = (String, Bool)

testResults :: [Test] → Bool
testResults = and . map snd
```

Problem:

A `Bool` does not convey how many or which tests failed.

Let’s Build a Test Runner

Using IO

```
type Test = (String, Bool)

runTests :: [Test] → IO ()
runTests = mapM runTest
  where
    runTest :: Test → IO ()
    runTest (name, res) = if res
      then putStrLn ("  ok   " ++ name)
      else putStrLn ("  FAIL " ++ name)
```

Exercise: Extend our Test Runner

Make it print either “All tests succeeded” or “*n* test(s) failed”.

```
type Test = (String, Bool)

runTests :: [Test] → IO ()
runTests = mapM runTest
  where
    runTest :: Test → IO ()
    runTest (name, res) = if res
      then putStrLn ("  ok   " ++ name)
      else putStrLn ("  FAIL " ++ name)
```



An Interactive Game

Wordley

Part 1

```
green, yellow, reset :: String -- ANSI text color escape chars
green  = "\ESC[42m"
yellow = "\ESC[43m"
reset  = "\ESC[0m"

wordley :: IO ()
wordley = do
  putStrLn ( "Let's play! Provide a number between 0 and "
           ++ show (length options) ++ " to play." )
  i ← getLine
  loop 6 (options !! read i)
```

Wordley

Part 2

```
loop :: Int → String → IO ()
loop n word
  | n ⩽ 0 = putStrLn "Game over!"
  | n > 0 = do
      putStrLn ("You have " ++ show n ++ " guess(es) remaining.")
      putStrLn "Enter a guess (5 characters):"
      guess ← getLine
      if guess == word
        then do putStrLn ("You got it in " ++ show (n - 5) ++ " guess(es)!")
                putStrLn ("The word was: " ++ green ++ word ++ reset)
        else do putStrLn "The game continues."
                mapM putStr [match g w word | (g,w) ← zip guess word]
                putStr "\n"
                loop (n - 1) word

match :: Char → Char → String → String
match g w word
  | g == w         = green  ++ [g] ++ reset
  | g `elem` word = yellow ++ [g] ++ reset
  | otherwise      = [g]
```

Learning Objectives

- ✓ Explain what a value of type `IO a` represents, and why Haskell programs stay pure despite performing input/output.
- ✓ Sequence actions using **`do`** notation, and explain how results flow from one line of a **`do`** block to the next.
- ✓ Use `pure` (or `return`) to inject a pure value into an action.
- ✓ Write a recursive interactive loop that carries state from iteration to iteration.
- ✓ Distinguish `map` from `mapM_` on a list of actions.